Attention for Dummies

Brian Cheng
ECE 539 LLMs - Presentation 1

Transformer Architecture

- 2 Main Components:
  - Encoder (left)
  - Decoder (right)

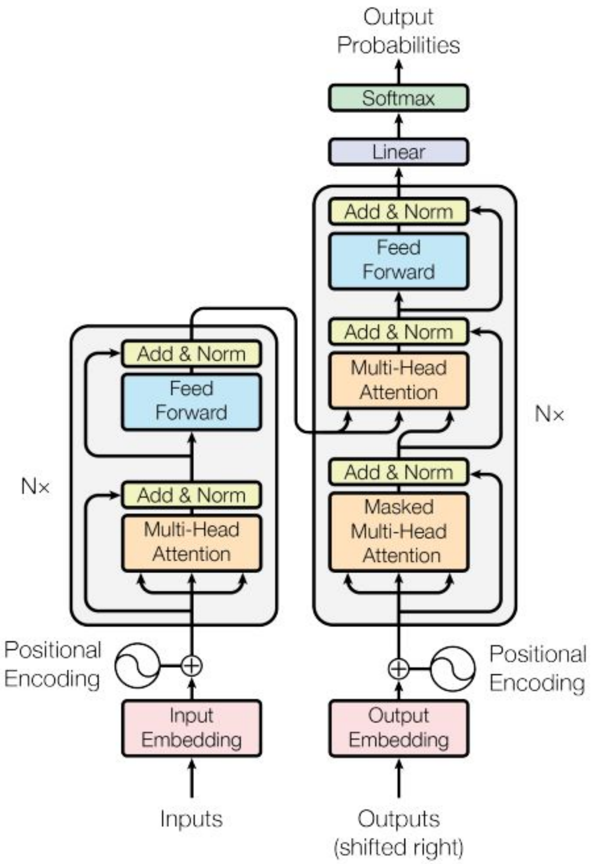


Figure 1: The Transformer - model architecture.

Encoder

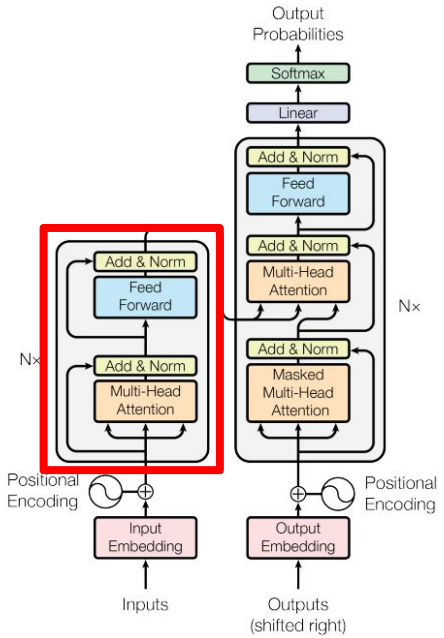
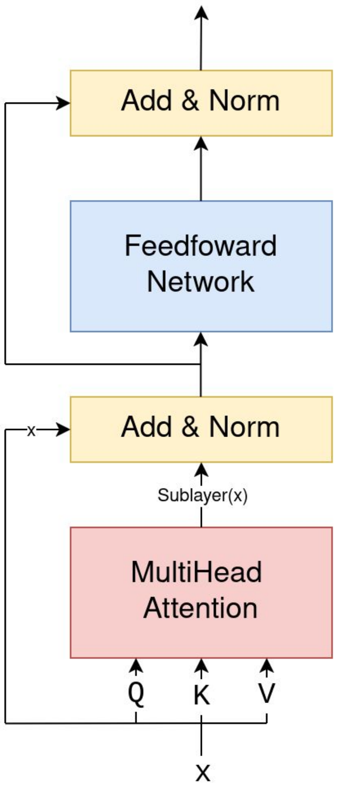


Figure 1: The Transformer - model architecture.



$$LayerNorm(x + Sublayer(x))$$

$$FFN(x) = max(0, xW_1 + b_1)W_2 + b_2$$

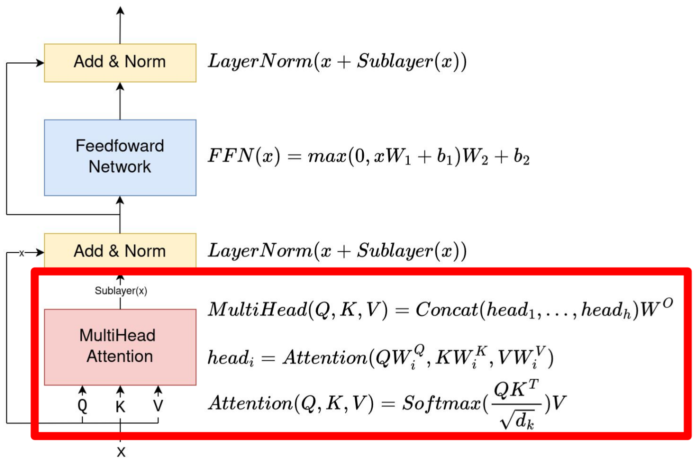
$$LayerNorm(x + Sublayer(x))$$

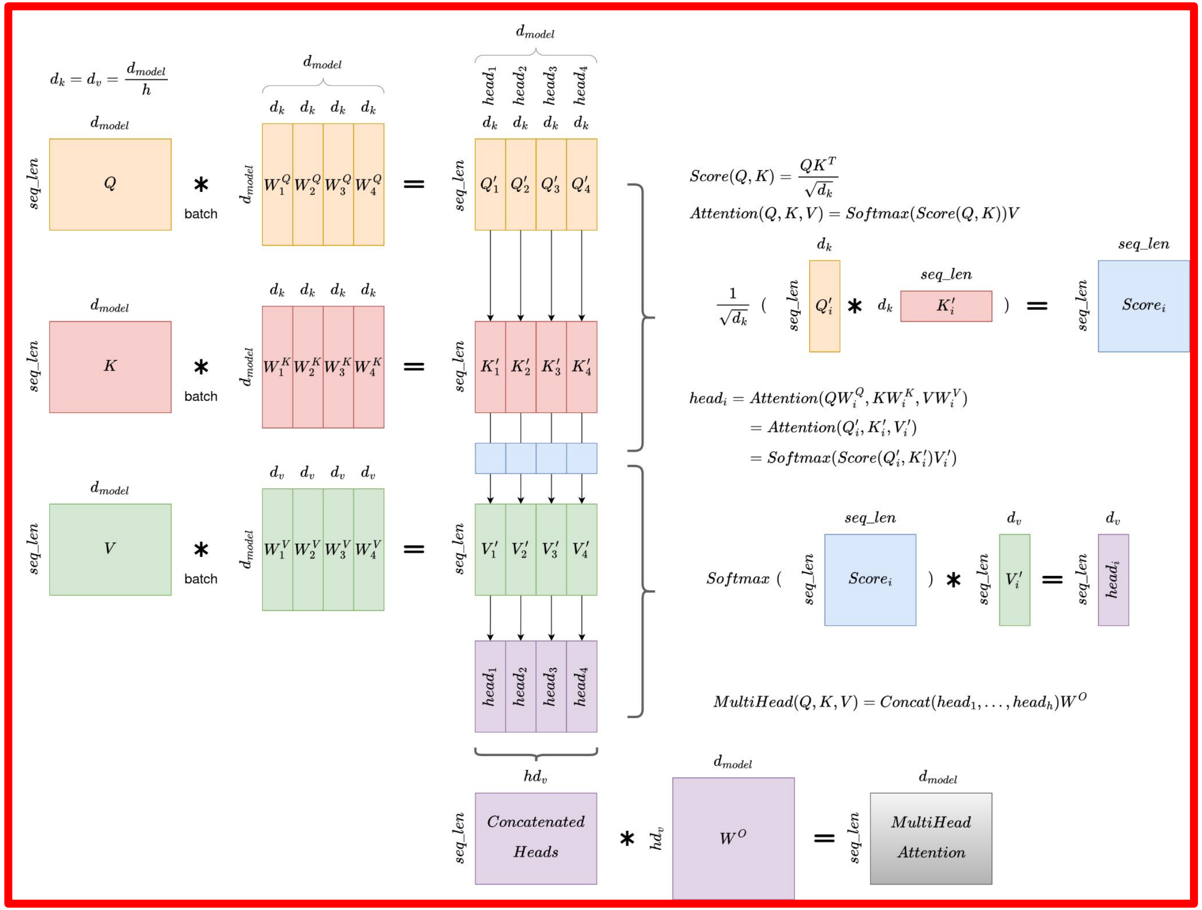
$$MultiHead(Q, K, V) = Concat(head_1, \ldots, head_h)W^O$$

$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$$

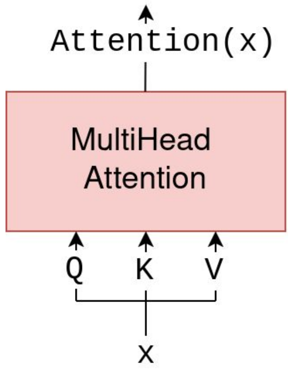
$$Attention(Q, K, V) = Softmax(\frac{QK^T}{\sqrt{d_k}})V$$

MultiHead Attention





Single Head Self-Attention

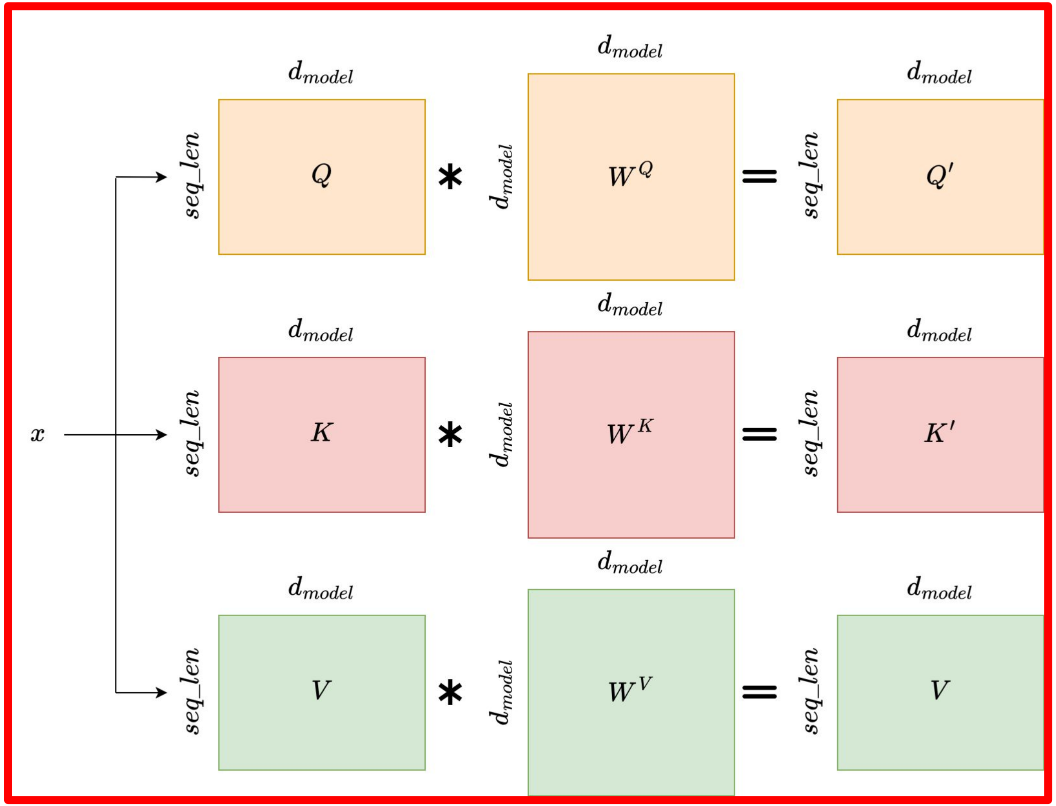


Self-Attention: Q, K, and V are the same input sequence x!

$$Attention(Q', K', V') = Softmax(Score(Q', K'))V'$$

$$\boxed{\begin{aligned} Q' &= QW^Q \\ K' &= KW^K \\ V' &= VW^V \end{aligned}}$$

$$Score(Q', K') = \frac{Q'K'^T}{\sqrt{d_k}}$$



Single Head Self-Attention

$$Attention(Q',K',V') = Softmax(Score(Q',K'))V'$$

$$Q' = QW^Q$$

$$K' = KW^K$$

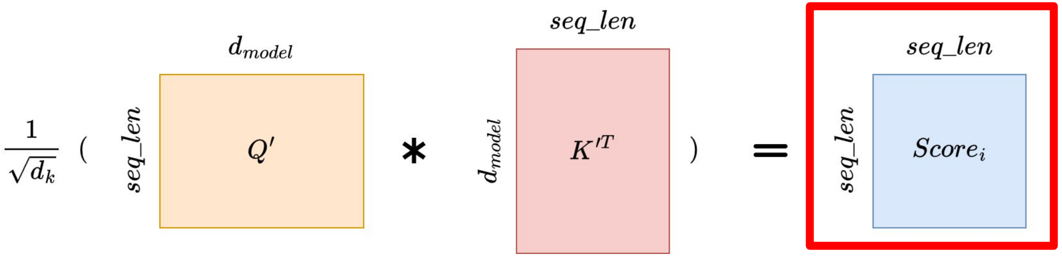
$$V' = VW^V$$

$$\boxed{Score(Q',K') = \frac{Q'K'^T}{\sqrt{d_k}}}$$

Query - which token is **looking** for context

Key - which token is **being looked at** by the query

A matrix of dot products between token embeddings.

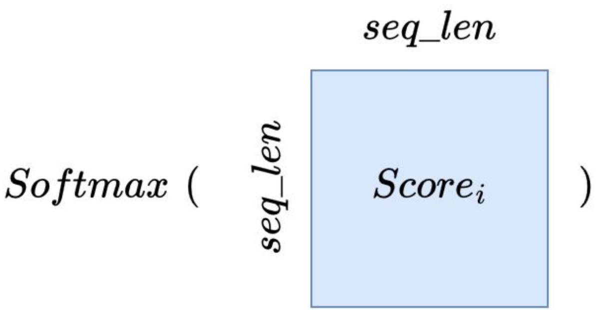


Queries

Keys	YOUR	CAT	IS	A	LOVELY	CAT
YOUR	0.268	0.119	0.134	0.148	0.179	0.152
CAT	0.124	0.278	0.201	0.128	0.154	0.115
IS	0.147	0.132	0.262	0.097	0.218	0.145
A	0.210	0.128	0.206	0.212	0.119	0.125
LOVELY	0.146	0.158	0.152	0.143	0.227	0.174
CAT	0.195	0.114	0.203	0.103	0.157	0.229

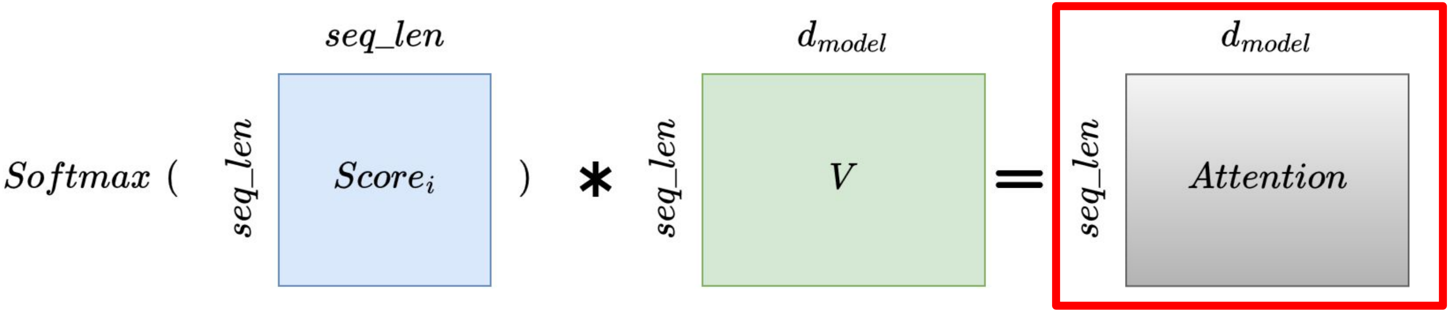
Single Head Self-Attention

Softmax: Take the raw scores, normalize each row such that the sum of each row equals one.



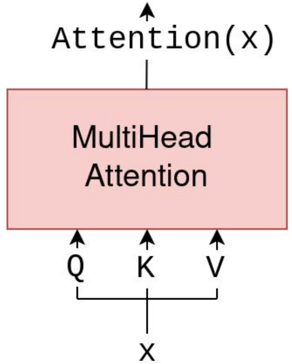
	YOUR	CAT	IS	A	LOVELY	CAT	Σ
YOUR	0.268	0.119	0.134	0.148	0.179	0.152	1
CAT	0.124	0.278	0.201	0.128	0.154	0.115	1
IS	0.147	0.132	0.262	0.097	0.218	0.145	1
A	0.210	0.128	0.206	0.212	0.119	0.125	1
LOVELY	0.146	0.158	0.152	0.143	0.227	0.174	1
CAT	0.195	0.114	0.203	0.103	0.157	0.229	1

Single Head Final Attention Scores



Captures the **meaning** and **position** of each token in the input sequence AND their **relationships** to all other tokens in the **vocabulary**.
A "contextualized embedding" for each token.

Single Head Attention Altogether

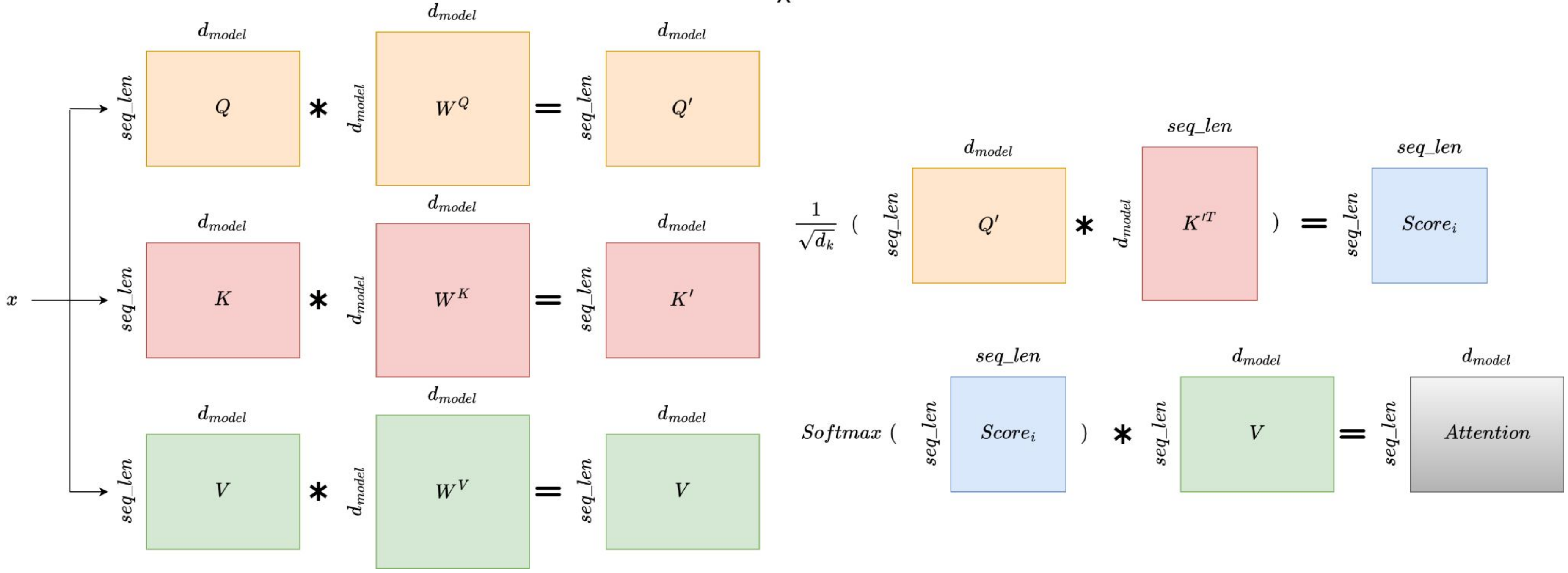


$$Attention(Q', K', V') = Softmax(Score(Q', K'))V'$$

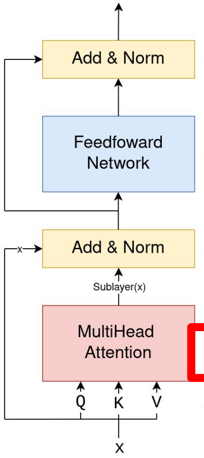
$$Q' = QW^Q$$

$$K' = KW^K \qquad Score(Q', K') = \frac{Q'K'^T}{\sqrt{d_k}}$$

$$V' = VW^V$$



MultiHead Attention



$LayerNorm(x + Sublayer(x))$

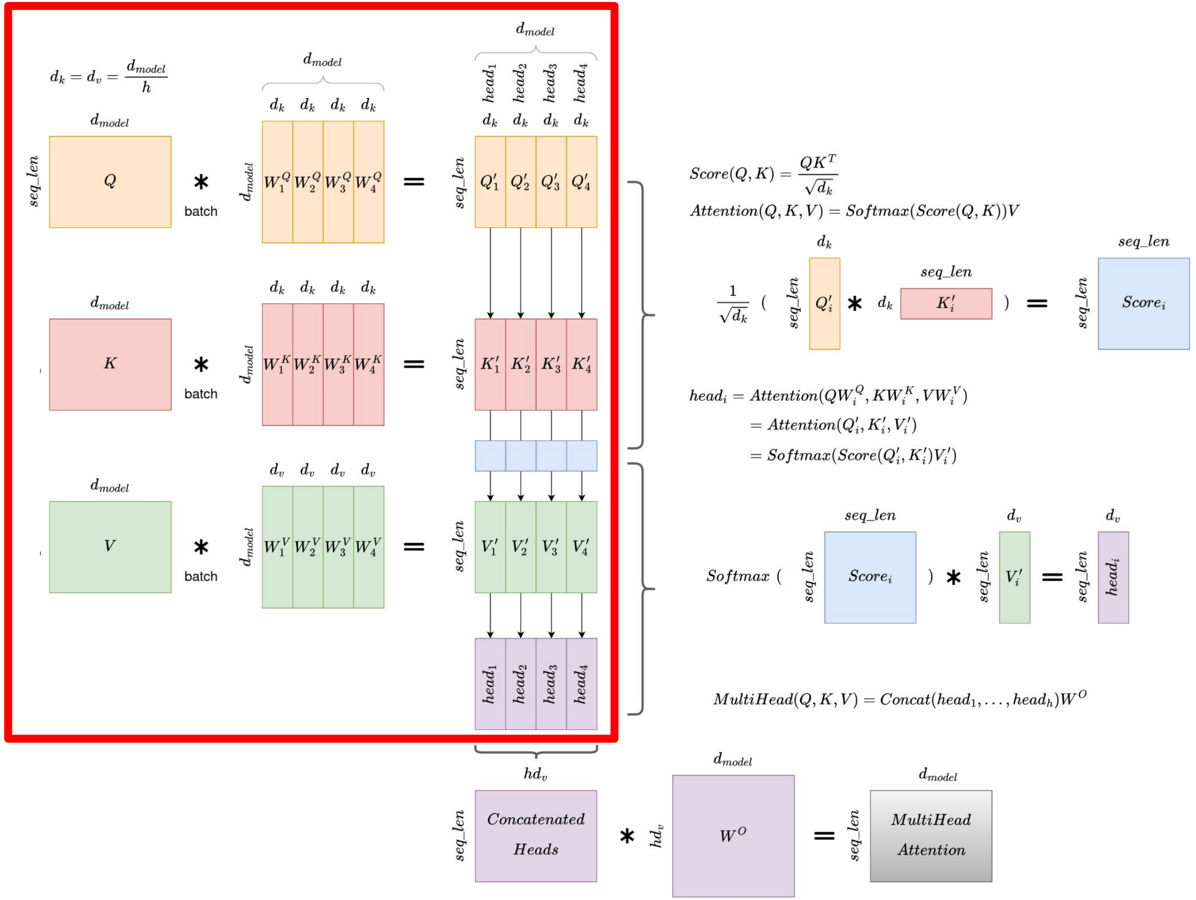
$FFN(x) = max(0, xW_1 + b_1)W_2 + b_2$

$LayerNorm(x + Sublayer(x))$

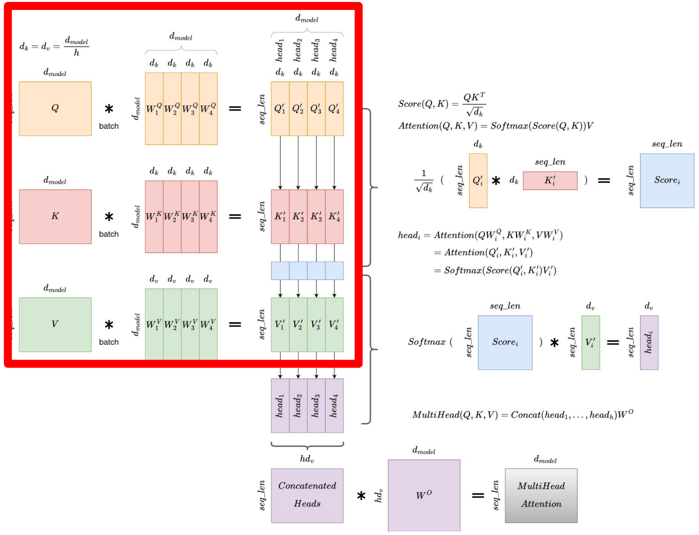
$MultiHead(Q, K, V) = Concat(head_1, \dots, head_h)W^O$

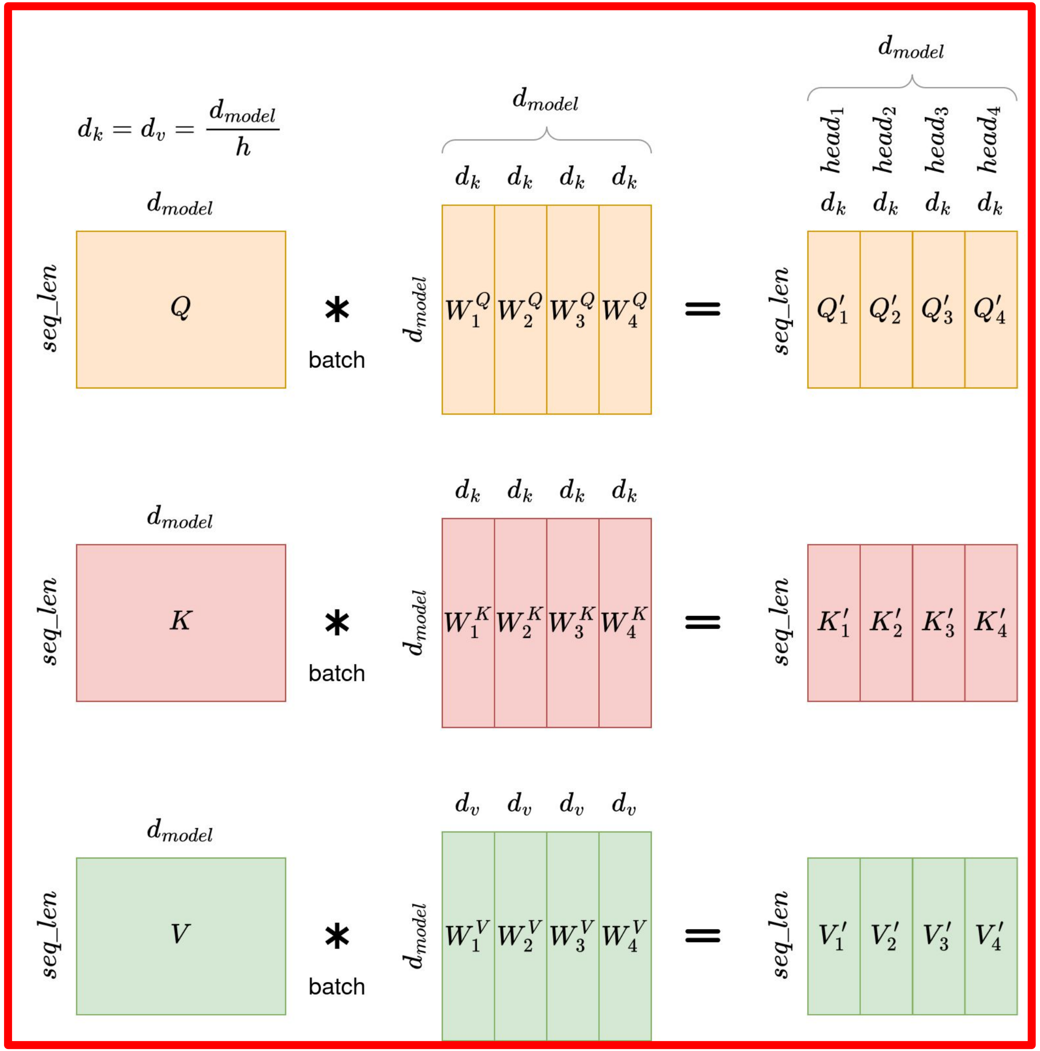
$\boxed{head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)}$

$Attention(Q, K, V) = Softmax(\frac{QK^T}{\sqrt{d_k}})V$



MultiHead Attention: Q' K' V'

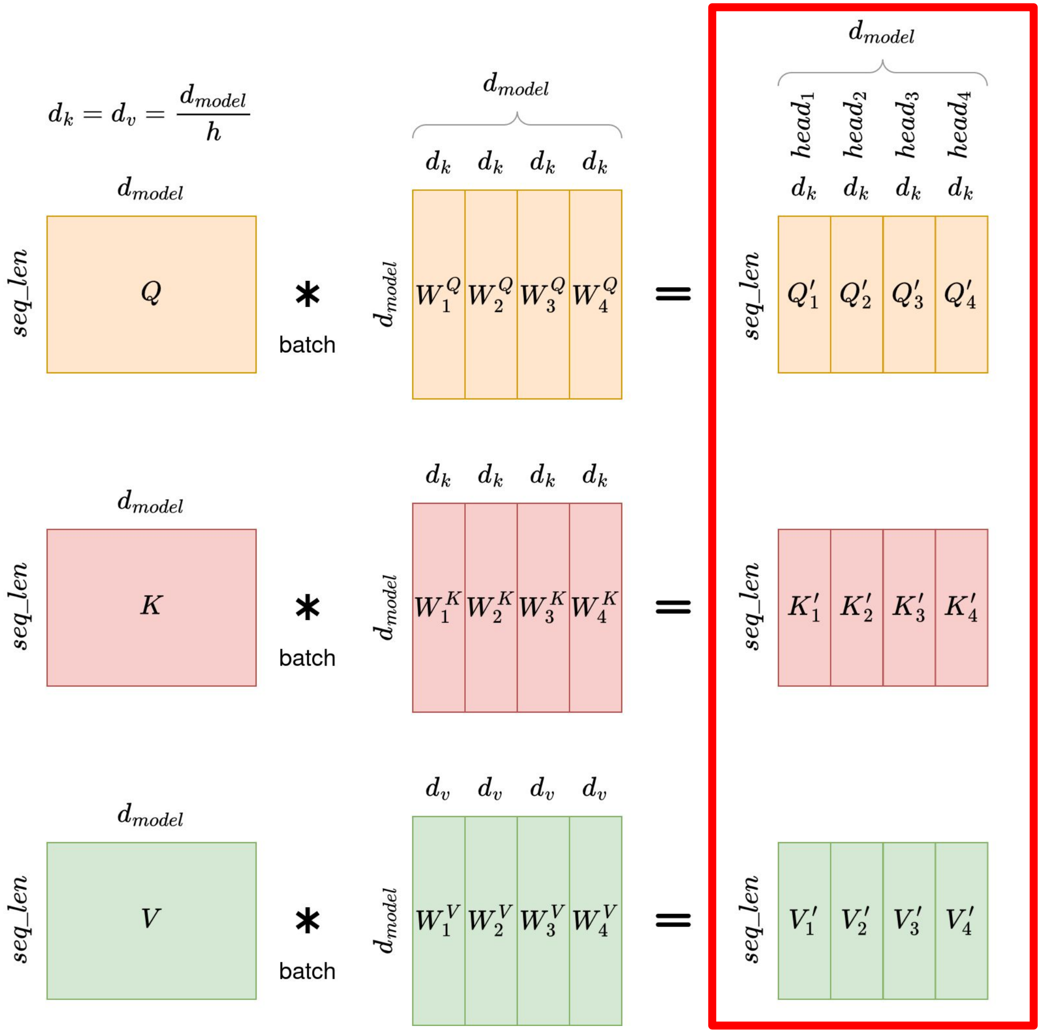


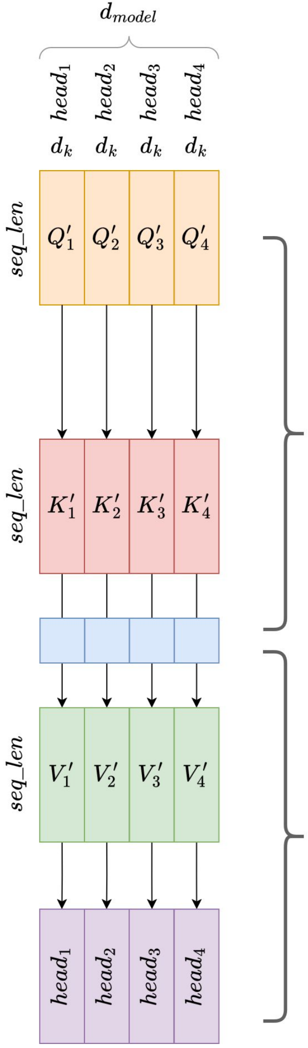


$$
\begin{aligned}
head_i &= Attention(QW_i^Q, KW_i^K, VW_i^V) \\
&= Attention(Q'_i, K'_i, V'_i)
\end{aligned}
$$

MultiHead Attention: Q' K' V'

$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V) = Attention(Q_i', K_i', V_i')$$

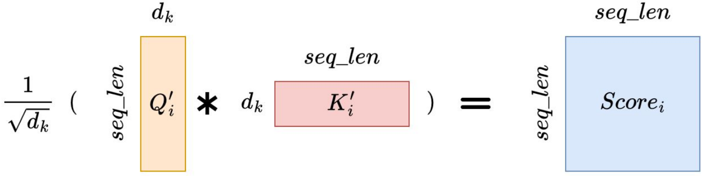




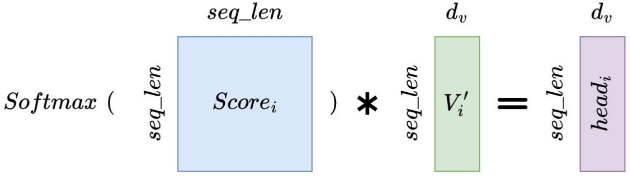
Calculate the Attention scores for each head.

$$Score(Q, K) = \frac{QK^T}{\sqrt{d_k}}$$

$$Attention(Q, K, V) = Softmax(Score(Q, K))V$$

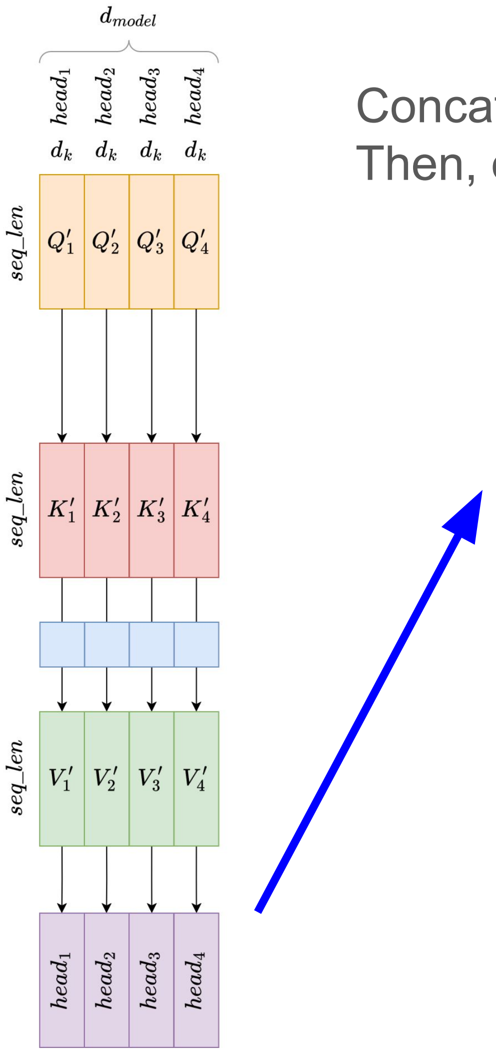


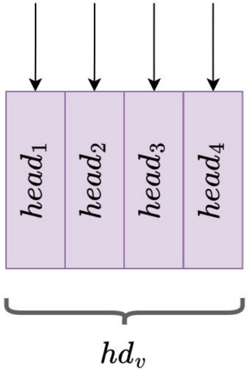
$$\begin{aligned} head_i &= Attention(QW_i^Q, KW_i^K, VW_i^V) \\ &= Attention(Q'_i, K'_i, V'_i) \\ &= Softmax(Score(Q'_i, K'_i)V'_i) \end{aligned}$$



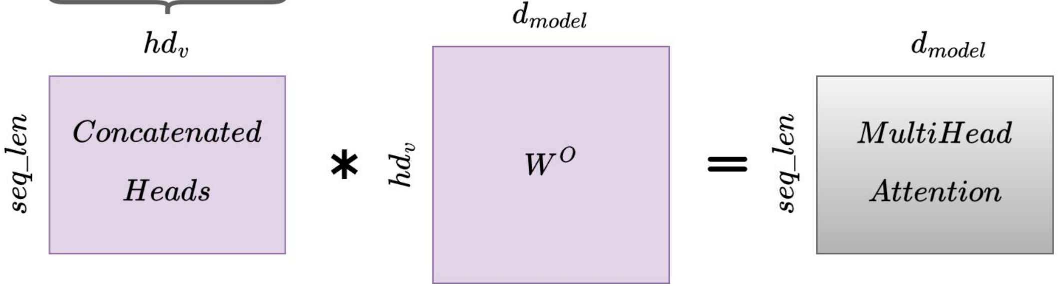
$$MultiHead(Q, K, V) = Concat(head_1, \ldots, head_h)W^O$$

Concatenate the heads back together.
Then, one final linear transformation with weight matrix W^O.

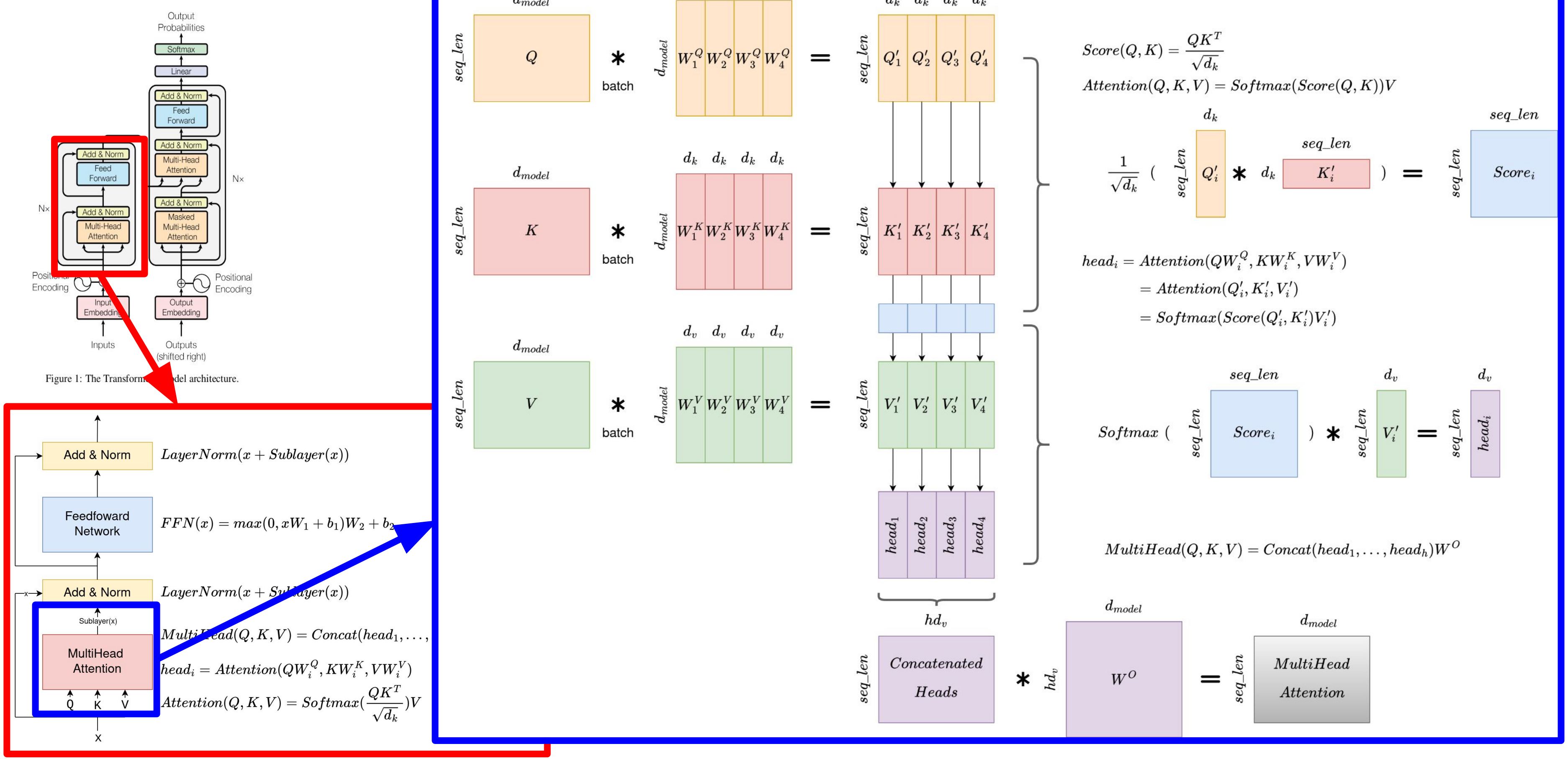




$$MultiHead(Q, K, V) = Concat(head_1, \ldots, head_h)W^O$$



Zooming out again…



Module Reuse

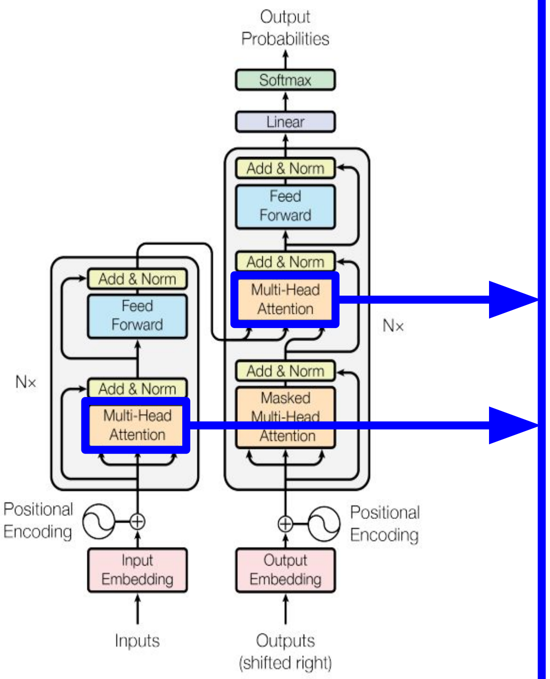
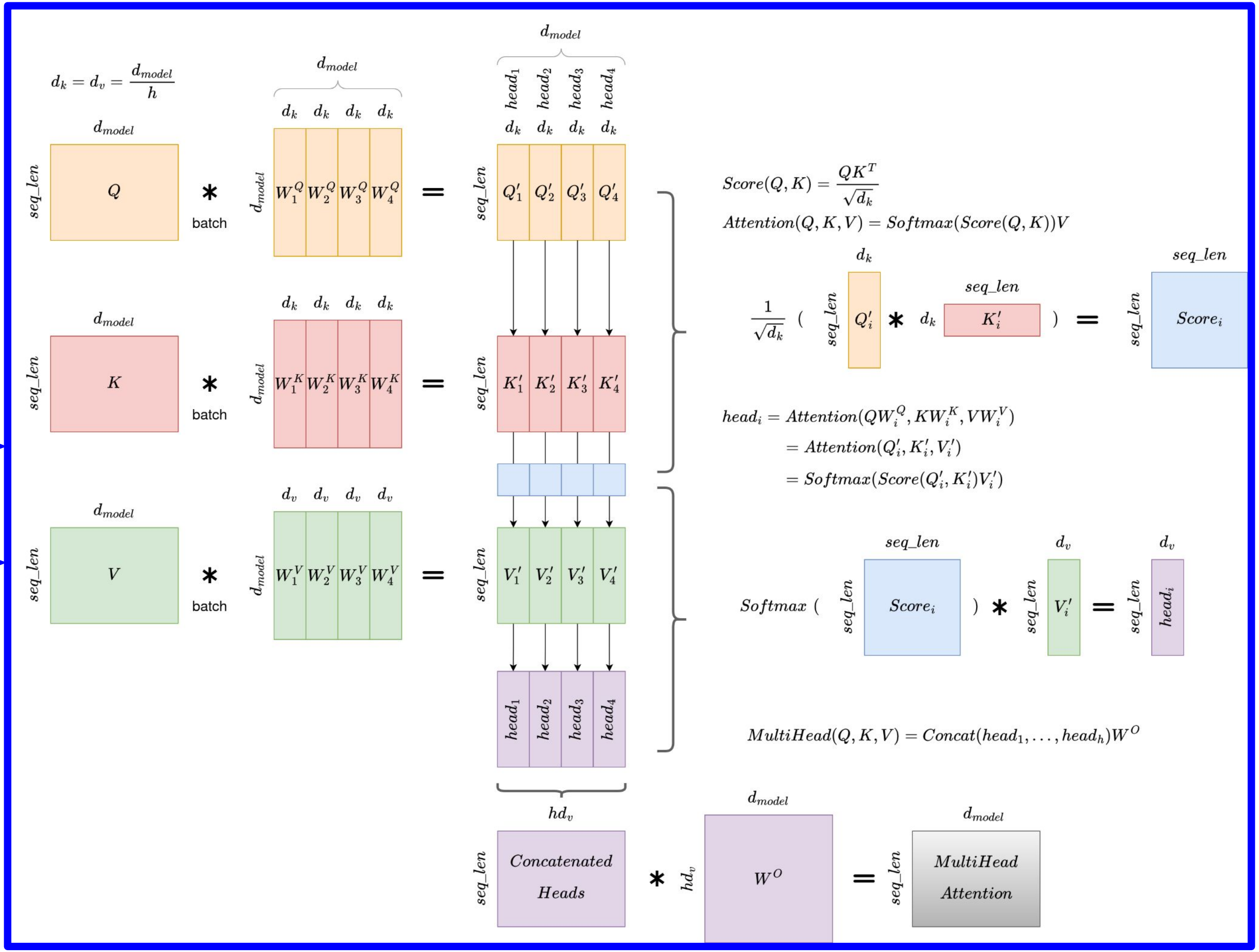


Figure 1: The Transformer - model architecture.



Masked MultiHead Attention (1/2)

What is “Masked” MultiHead Attention?

It’s just the same MultiHead Attention module but with one extra step in the middle!

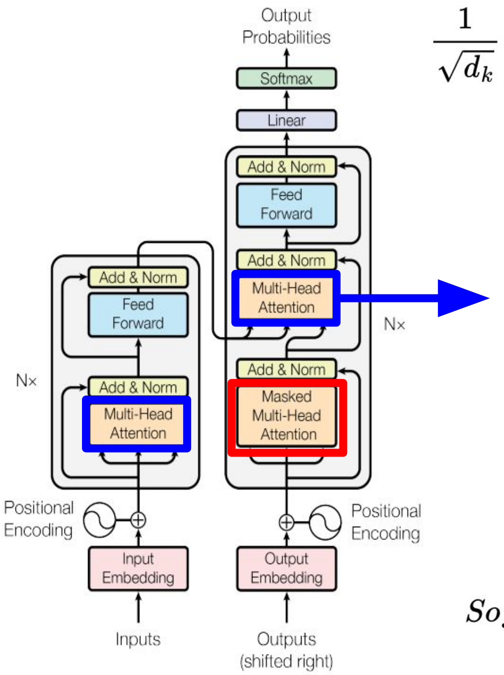
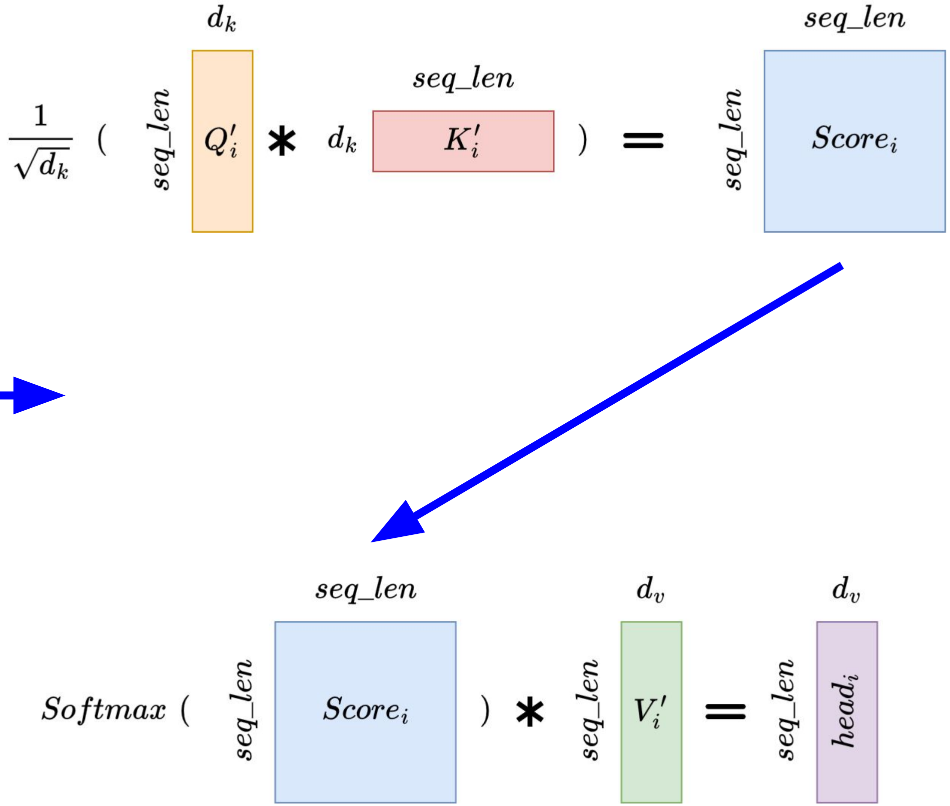


Figure 1: The Transformer - model architecture.



Masked MultiHead Attention (2/2)

What is “Masked” MultiHead Attention?

It’s just the same MultiHead Attention module but with one extra step in the middle!

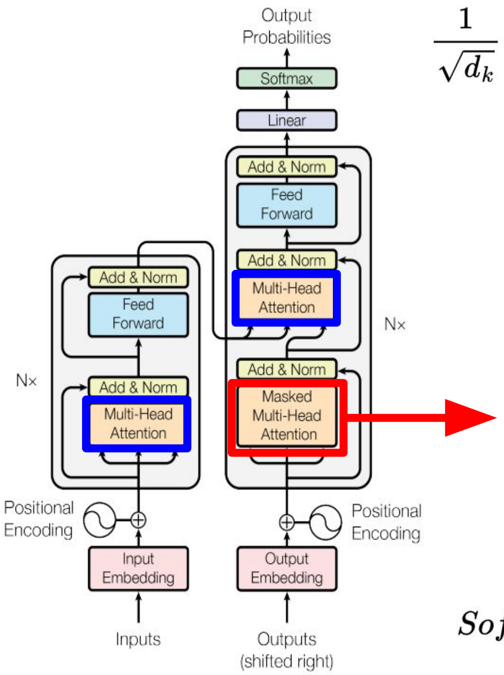
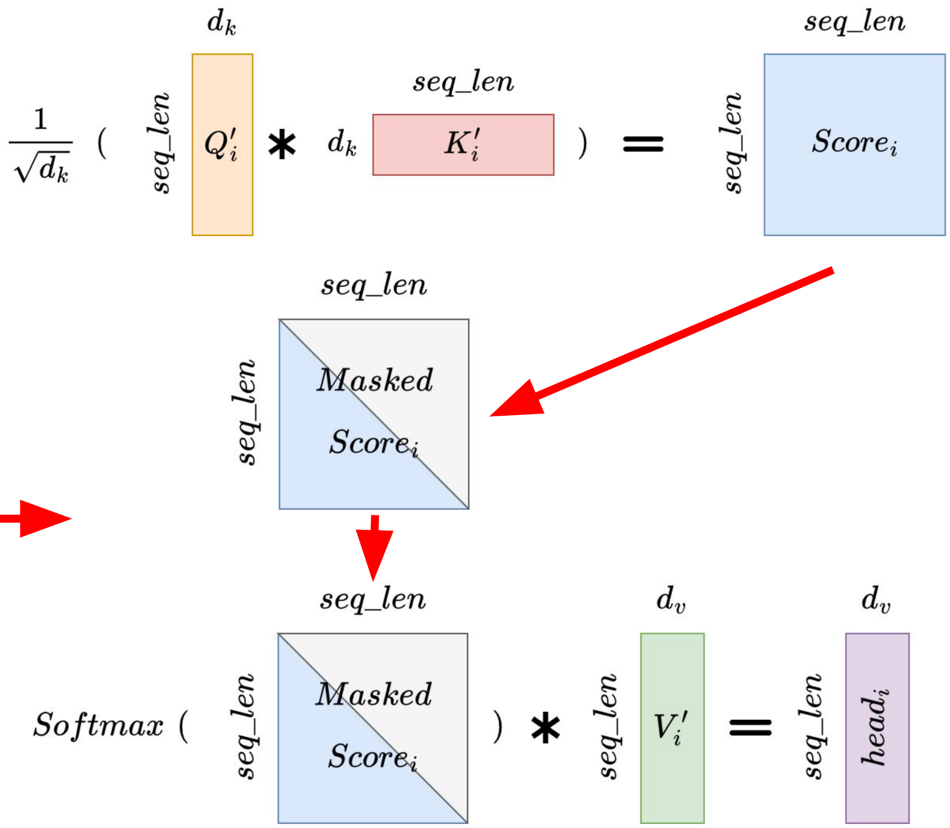
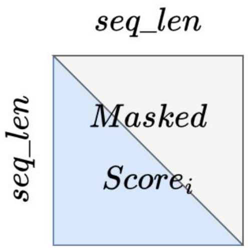


Figure 1: The Transformer - model architecture.



What is the Mask? (1/2)



Pass the lower triangular half (diagonal inclusive) of the input matrix.

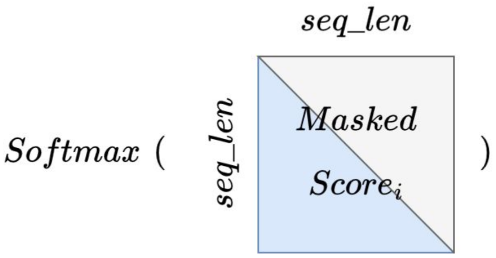
Set the upper triangular half to minus infinity, such that taking the softmax results in zero for that score.

Keys

Queries

	YOUR	CAT	IS	A	LOVELY	CAT
YOUR	0.268	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$
CAT	0.124	0.278	$-\infty$	$-\infty$	$-\infty$	$-\infty$
IS	0.147	0.132	0.262	$-\infty$	$-\infty$	$-\infty$
A	0.210	0.128	0.206	0.212	$-\infty$	$-\infty$
LOVELY	0.146	0.158	0.152	0.143	0.227	$-\infty$
CAT	0.195	0.114	0.203	0.103	0.157	0.229

What is the Mask? (2/2)



Pass the lower triangular half (diagonal inclusive) of the input matrix.

Set the upper triangular half to minus infinity, such that taking the softmax results in zero for that score.

Keys

Queries

	YOUR	CAT	IS	A	LOVELY	CAT
YOUR	0.268	0	0	0	0	0
CAT	0.124	0.278	0	0	0	0
IS	0.147	0.132	0.262	0	0	0
A	0.210	0.128	0.206	0.212	0	0
LOVELY	0.146	0.158	0.152	0.143	0.227	0
CAT	0.195	0.114	0.203	0.103	0.157	0.229

Softmax(-∞) = 0

Why mask?

To ensure causality:
The output at a certain position of the input sequence can only depend on the tokens of the previous positions.
The model **must not** be able to see future words at any given position.

Keys

Queries	YOUR	CAT	IS	A	LOVELY	CAT
YOUR	0.268	0	0	0	0	0
CAT	0.124	0.278	0	0	0	0
IS	0.147	0.132	0.262	0	0	0
A	0.210	0.128	0.206	0.212	0	0
LOVELY	0.146	0.158	0.152	0.143	0.227	0
CAT	0.195	0.114	0.203	0.103	0.157	0.229

Recap (1/3)

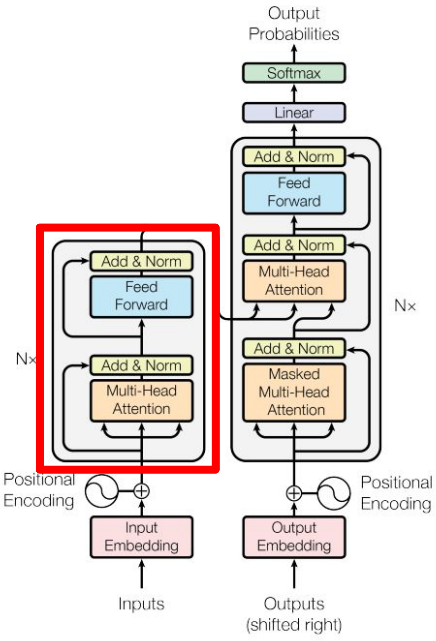
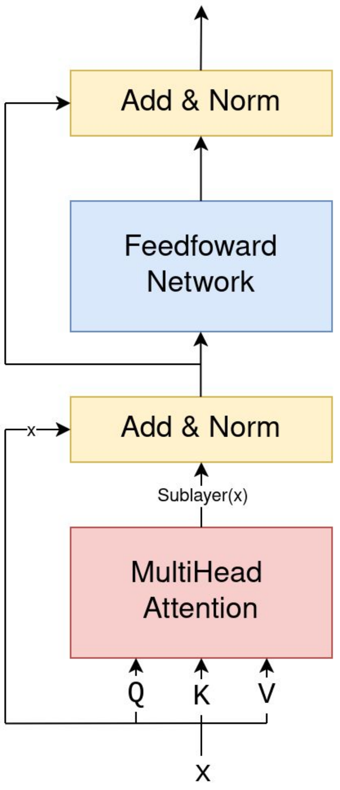


Figure 1: The Transformer - model architecture.



$$LayerNorm(x + Sublayer(x))$$

$$FFN(x) = max(0, xW_1 + b_1)W_2 + b_2$$

$$LayerNorm(x + Sublayer(x))$$

$$MultiHead(Q, K, V) = Concat(head_1, \ldots, head_h)W^O$$

$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$$

$$Attention(Q, K, V) = Softmax(\frac{QK^T}{\sqrt{d_k}})V$$

Recap (2/3)

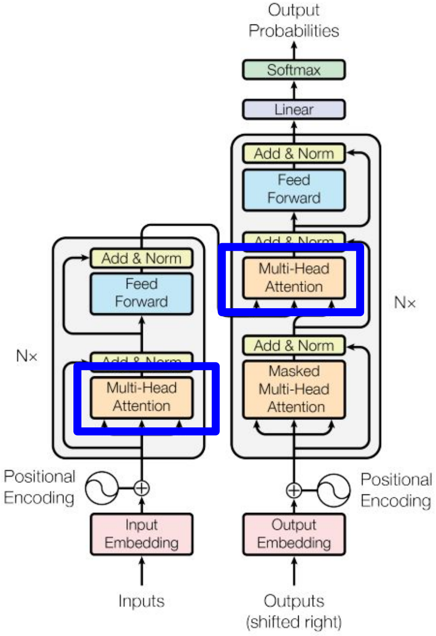
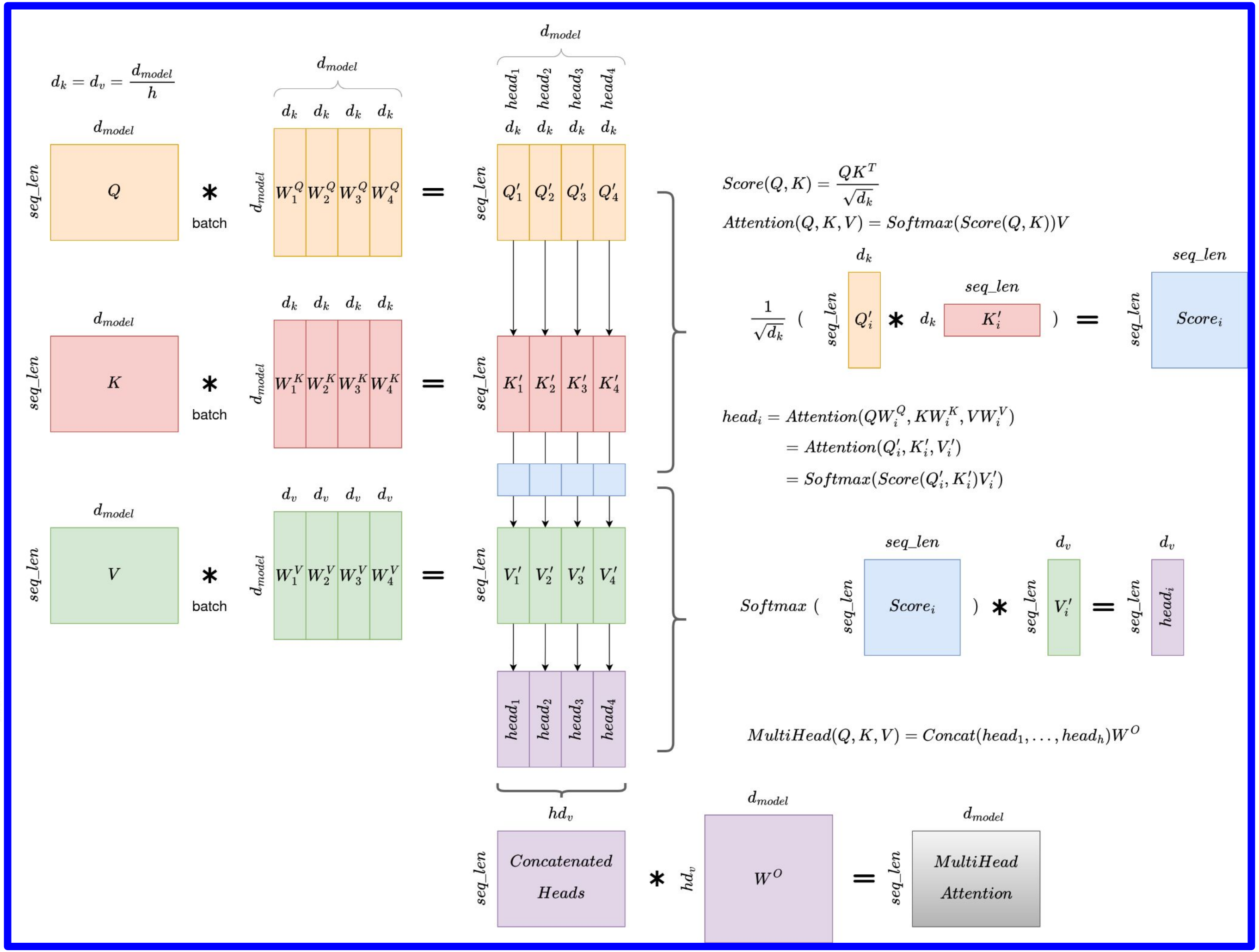


Figure 1: The Transformer - model architecture.



Recap (3/3)

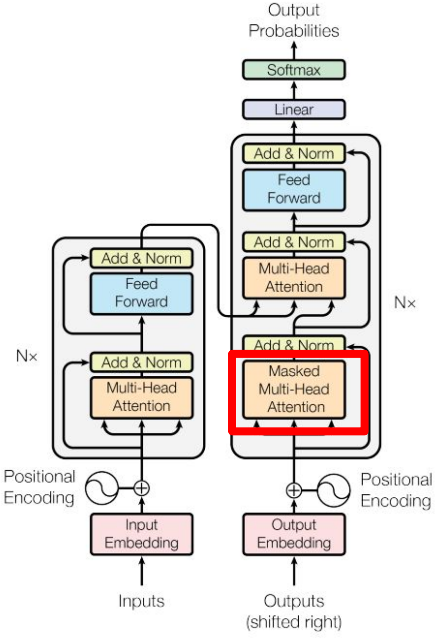


Figure 1: The Transformer - model architecture.